



Islamic University of Technology (IUT)

Department of Electrical and Electronic Engineering

A subsidiary organ of the OIC

Project File

EEE 4308: Digital Electronics Lab

Name of the Project:

Digital Snake Ladder Game

Phase : C, D & E

Phase-C: (i) Dice Module

(ii) High Frequency Memory Clock (using 555 Timer IC)

Phase-D: (i) Memory Module

(ii) Reset Module

(iii) Player Selector Module

(iv) Winner Indicator Module

Phase-E: Complete Integration of Digital Snake Ladder Game

Group Name: Trailblazers

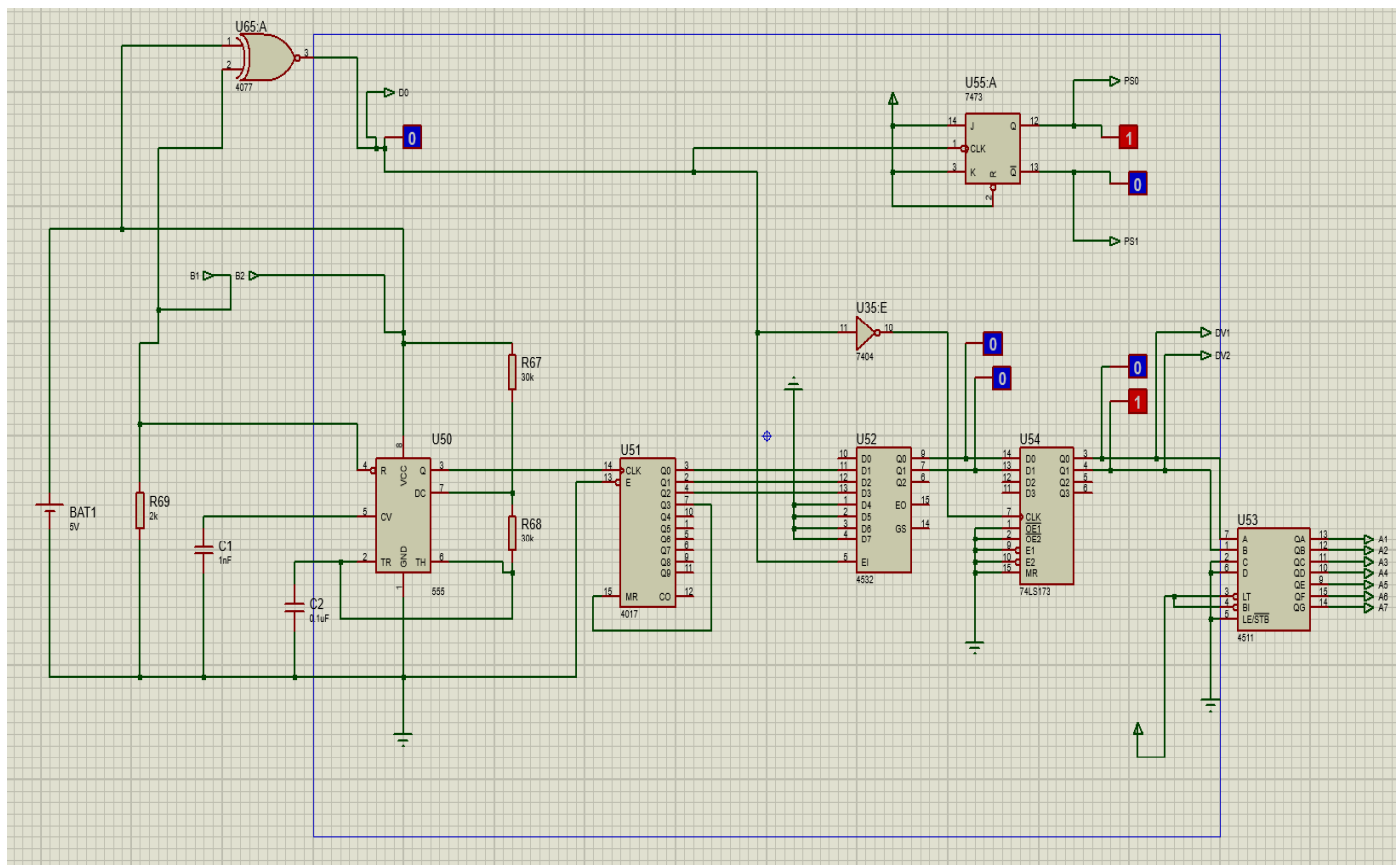
Student Name	Student ID
1. Md. Abid Sarwar Khan	220021122
2. Md. Tanzim Ashfaq	220021126
3. Nowrin Islam Nishat	220021147
4. Rafjanul Alam Rafi	220021153
5. Munim Shahrier	220021154
6. Umme Suraia Jannati	220021158

Problem Statement of Phase- C, D & E:

In our phase C, we have create a dice module which will determine the numbers obtained by each player when the dice is rolled by each player. We also have to use 555 Timer IC in order to adjust our frequency by varying resistors and capacitors when needed. Again, in phase D, we have to build reset module, player selector module and the all important memory module where the positions of the players will be stored. Then, we have to build the winning module which will determine the final winner of the game depending on who finishes first and reaches to position 15 in the game board.

Phase-C

Dice Module & High Frequency Timer Clock:



In the dice module, we have used the high frequency timer clock where we can adjust our frequency by the help of capacitors and resistors which are connected through a battery source of 5V along with a switch. We alter the values of our resistors and capacitors to form a high frequency through a 555 Timer IC. From this 555 Timer IC, we find the frequency for our clock pulse in the 4017 IC known as Johnson Counter. As our frequency is high, we will see the the three outputs of our counter alternating at a very rapid speed . It'll then go to the priority encoder where our three possible results will enter as inputs with fast alternation due to the high frequency and we will get the output in 2-bit binary. The 2-bit binary will then proceed to the parallel in-parallel out 2-bit shift resistor using D flip flops and storing our data in the flip flops. When clock pulse is triggered this stored value goes to the register output which is our final dice value.To show that dice value we have connected the output to BCD to 7-segment decoder in order to display our dice value in our 7 segment display.

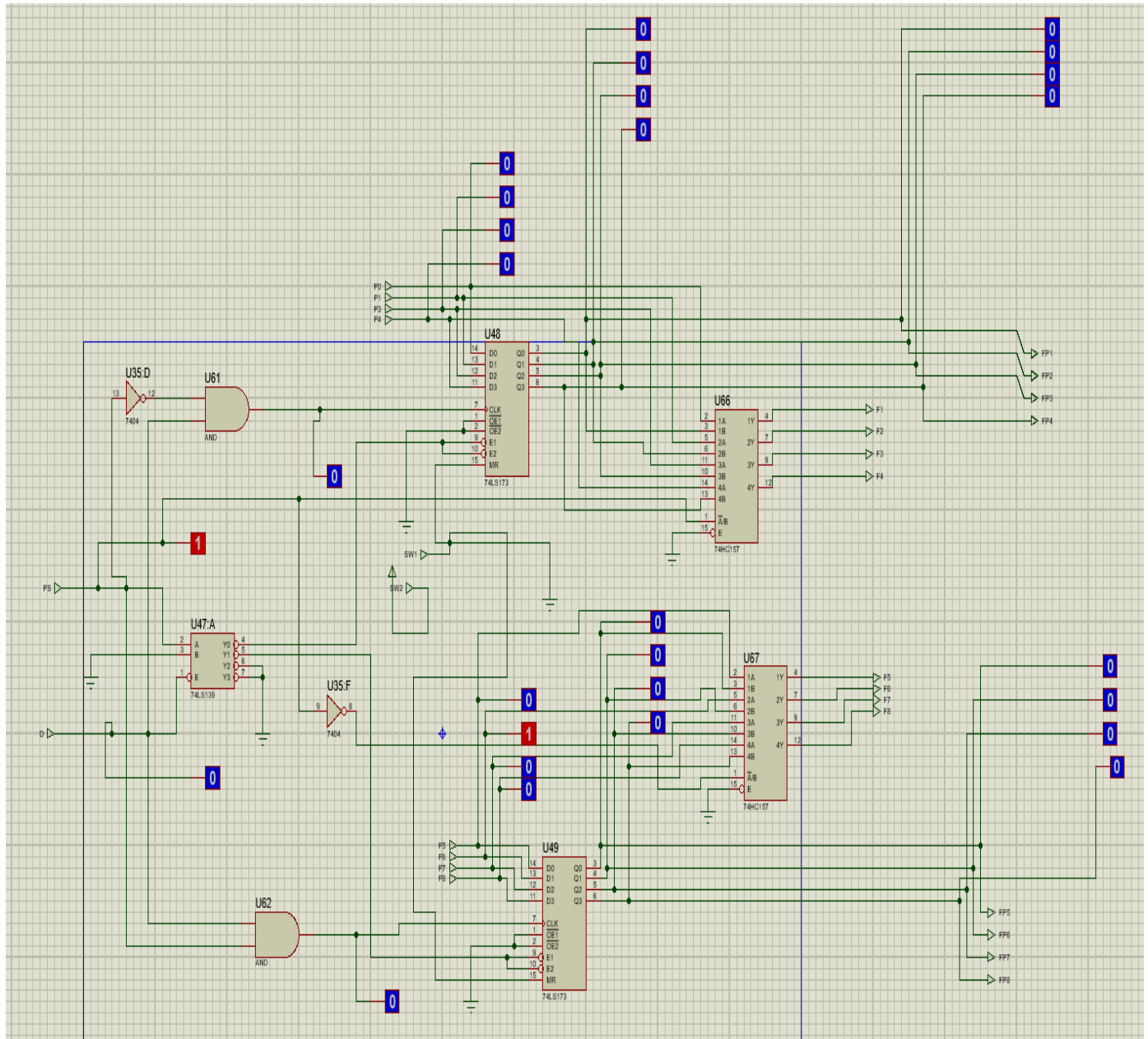
Phase-D

Memory Module:

The memory module is set up in such a way that whenever our switch button is pushed, the D power button enables our MUX which is used to determine that when one player is selected, other player must not operate. In that MUX, we have also given the input of player selection. So, when the MUX is enabled, the output of the MUX has been connected to two shift resistors for each player. When Player 1 is selected it goes through the MUX enables the upper shift resistor defined for player 1 and thus the other shift resistor stops operating at the same time. This happens vice versa for player 2 also.

Now, in the shift resistors, we are giving the output of our dice module as the input. However we saw a problem that for one clock pulse triggering, we were not seeing the output of the shift resistor for player 1's final dice module value which was supposed to show in the output and thus in the LED. Rather, the value was held in the D flip flop inputs of the resistors for each player. When another clock pulse triggering happens due to the push in our switch button, we see the output in shift resistor of player 1's dice module value where we were supposed to get the value of player 2's dice module value in the 2nd clock pulse triggering. So, as it stood out that our circuit was running one clock pulse behind in terms of showing the dice value for each player in the LED. To resolve this matter, we have used 2 more MUX for each player. In that MUX, we are giving connection to our "PLAYER SELECTOR (PS)". We are also connecting the input of D-flip flop directly to our MUX at A inputs and also connecting the output of shift resistor to the MUX at B inputs. So, when player 1 is selected for dice click, due to the active low operation of A,

inputs of D-flip flops connected directly to the MUX are transmitted to the LED'S. Whereas, when player 2 is selected, the inputs of 2nd shift resistor is directly transmitted through the 2nd MUX to the LED'S to show the output pf player 2. Updated values of the memory are connected to the adder circuit in our integrated circuit where the adder takes input of our dice value and memory value to count the new player's next position.



Player Selector Module:

In the player selector module, we have used JK flip flop. We have built our clock pulse in such a way that when switch button is pushed, clock pulse of the JK flip flop triggers. Now, in our JK flip flop, our inputs are always connected to the power supply ($J=K=1$) so that it toggles.

Primarily, we have fixed that player 1 is our blue player having output 0 and player 2 is our red player having output 1. So, when clock pulse triggers and due to the power on condition, in every triggering player selector toggles. At first, the blue player is selected as player 1 (output=0) and when triggering occurs player 2 is selected (output=1). In this way when we turn on the switch button, toggling occurs and each player is selected for dice roll one after another.

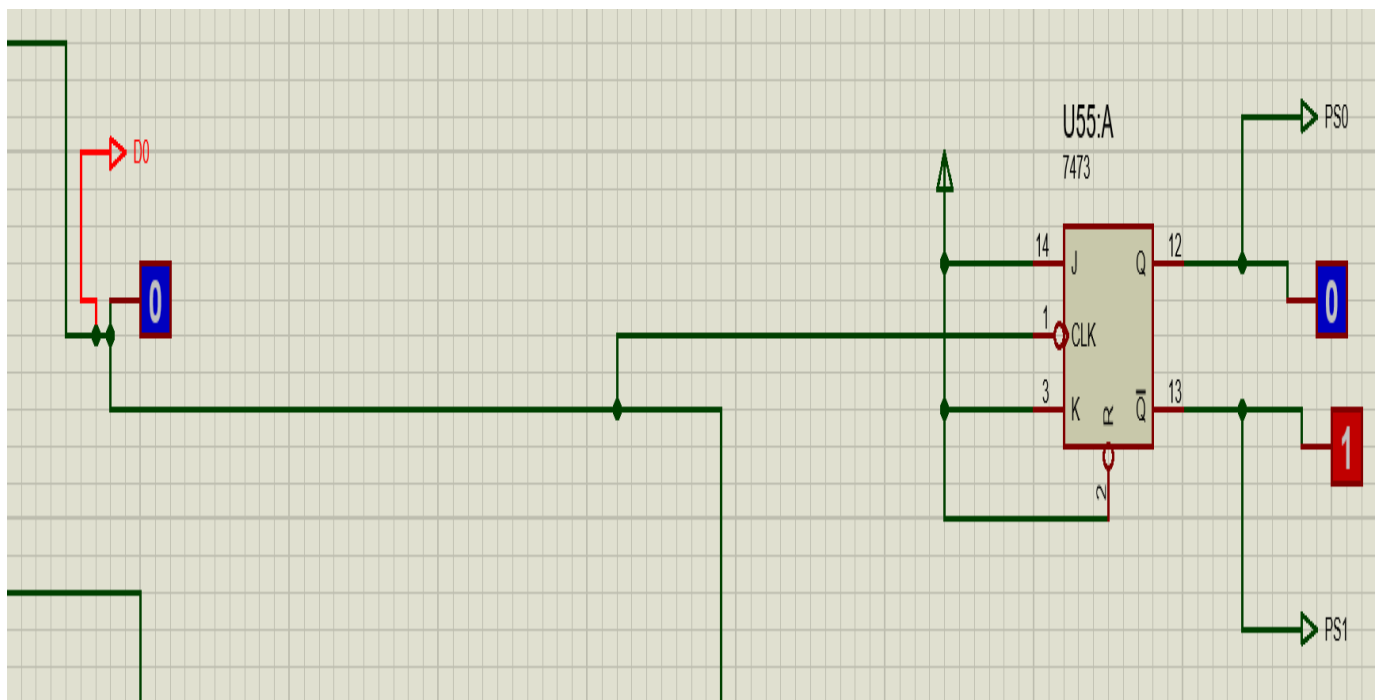


Fig: Player Selector Module with JK flip flop

Reset Module:

In our reset module, the MR button of parallel in parallel out shift resistor works as reset. When MR is connected to the ground, the game is running normally. However, we have used the SW-SPDT switch which works in both ways. The SW-SPDT switch is connected between power and ground such that when we click the switch, it will open circuit one path and short circuit the other one. So, in this way, when MR is connected to the ground, it will operate normally. But when the switch button is clicked MR gets open circuited with the ground and short circuits the path between power and shift resistor. When, the MR is short circuited with the power, the game resets immediately. So, all the data stored in the memory resets also.

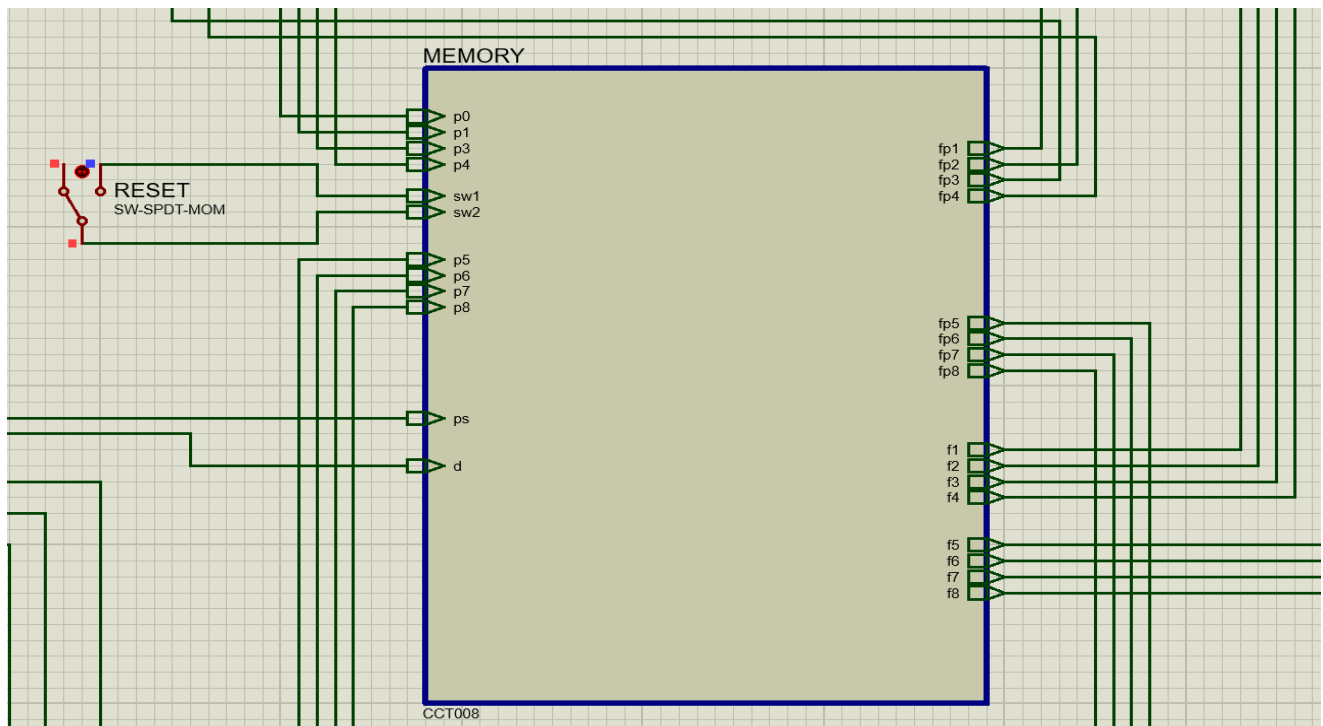


Fig: SW-SPDT reset switch

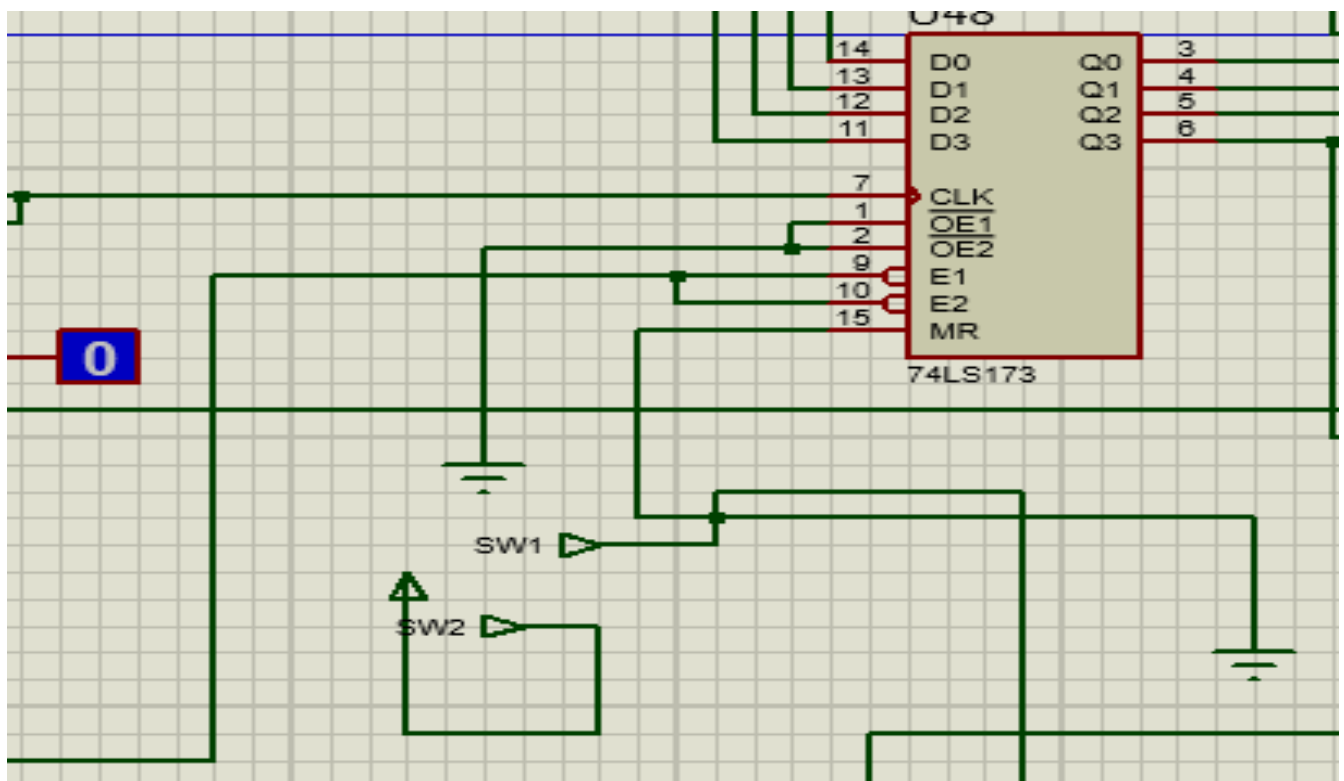


Fig: Reset module circuit

Winner Indicator Module:

In the winning module, we know that whoever reaches to 15 at first is the winner. So, we have connected our active low player LED's to the position 15 and connected those with a X-OR gate. From the output of that X-OR gate we have connected a buzzer. The other end of the buzzer is connected with the power source. Buzzer will ring and indicate the winning player LED. So, when a player reaches to 15 first, we will have a 1 and 0 input to our X-OR gate and from the operations of X-OR we will get output 1 as well as the LED of the player who reaches to 15 1st will light up and due to the output 1 of the X-OR and power of buzzer circuit, buzzer will ring for the player who has reached to 15. Thus, it indicates the player who wins the game by lighting up the corresponding LED of that player alongside with the buzzer.

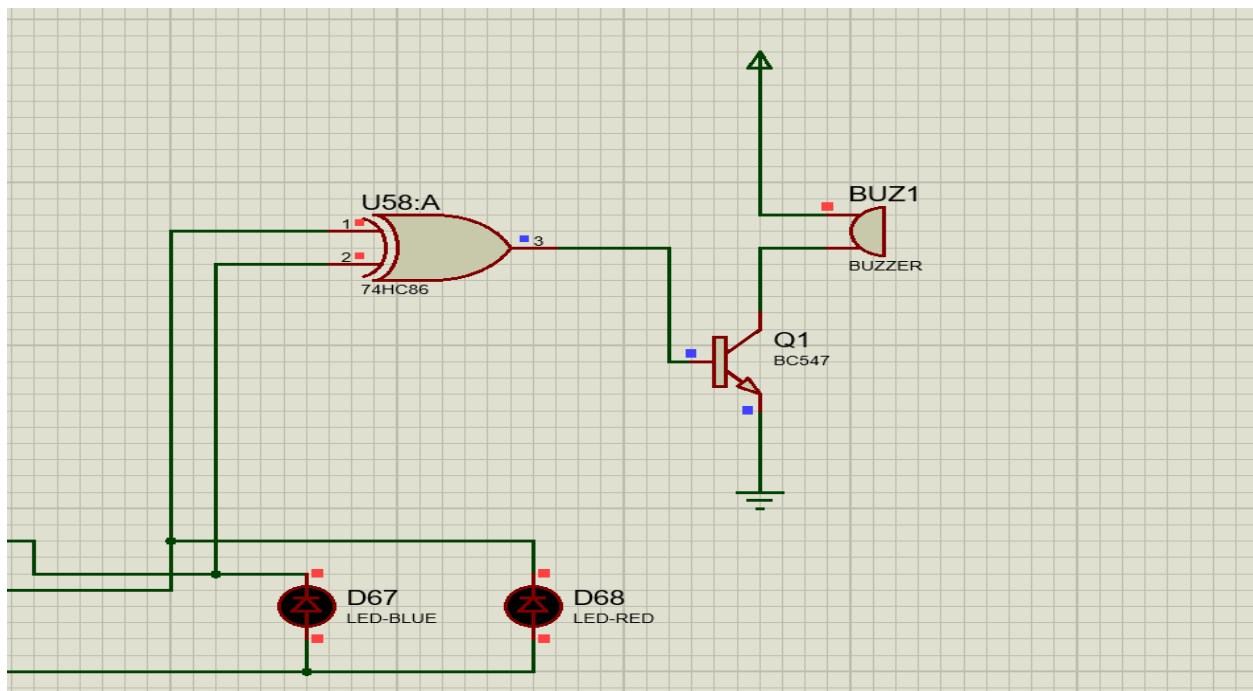


Fig: Winner Module Circuit with the Buzzer

Phase-E

Complete Integration of Digital Snake Ladder Game:

In the complete integrated circuit build up page of our game in proteus, we see that there are 2 adders just beside the dice module. These 2 adders are actually one for player 1 is another for player 2. We can also see there are 4 MUX. 2 MUX for each player again.

In the top right side of the dice module, adder and 2 MUX's are for player 1. In the bottom right side of the dice module, adder and 2 MUX are for player 2. The function of the adder is such that the 2 inputs are one from the dice module which is the value of a player's dice roll and another input comes from the memory where we had stored the positions of players. Now, the output of that adder indicates the current position player is supposed to obtain after dice roll. However, a special case occurs. For example, if a player is at position 14 of the game, then he will need 1 more point to win. But if the dice roll gives more than 1 then he must not proceed any further but stay where he was. He will only proceed to the last position if he obtains 1 in the dice roll.

Operation of MUX-1:

To apply the logic stated above, we have connected the carry out of our adder to the A'/B of the MUX. We have connected the adder outputs to the A input of the MUX which will respond to the active low input signal. Again, we have connected the

memory values directly to the B inputs of MUX which responds to active high signals. So, when the dice value after adding with the memory value of the players position is equal to or less than 15, there is normally no carry output. So, carry out will transfer 0 output to the MUX button A'/B . It will then take input of the A in our MUX and transmit it to another MUX. It means that if there is no carry then our dice value will add up with our stored value of players in the memory and thus transfer it through to the 2nd MUX.

However, if there is a carry which means dice value+stored value of memory is greater than 1 then the carry out will enable the B input of our 1st MUX which is directly the stored value of the memory. It'll then transfer the B input to the output of MUX-1 and then to the input of the 2nd MUX.

Operation of MUX-2 for each player:

MUX-2 basically works so that the circuits for both players doesn't operate at the same time. That's why we are connecting the button A'/B of the MUX with player selector (PS). As we know that, we fixed our player 1 as blue player with 0 output in and player 2 as red player with 1 output, so when player 1 rolls the dice, only the value player 1 is transmitted and the circuit of player 2 stops. Again, when player 2 rolls the dice , the value of player 2 must be transmitted but the circuit of player 1 must stop. In this way, the MUX-2 makes sure that both circuit of players don't operate at the same time. The output of both MUX-2 of both players then goes to the logic module where logic is

generated for 3 cases such as if the players come at the snake head position or ladder bottom or normal case.

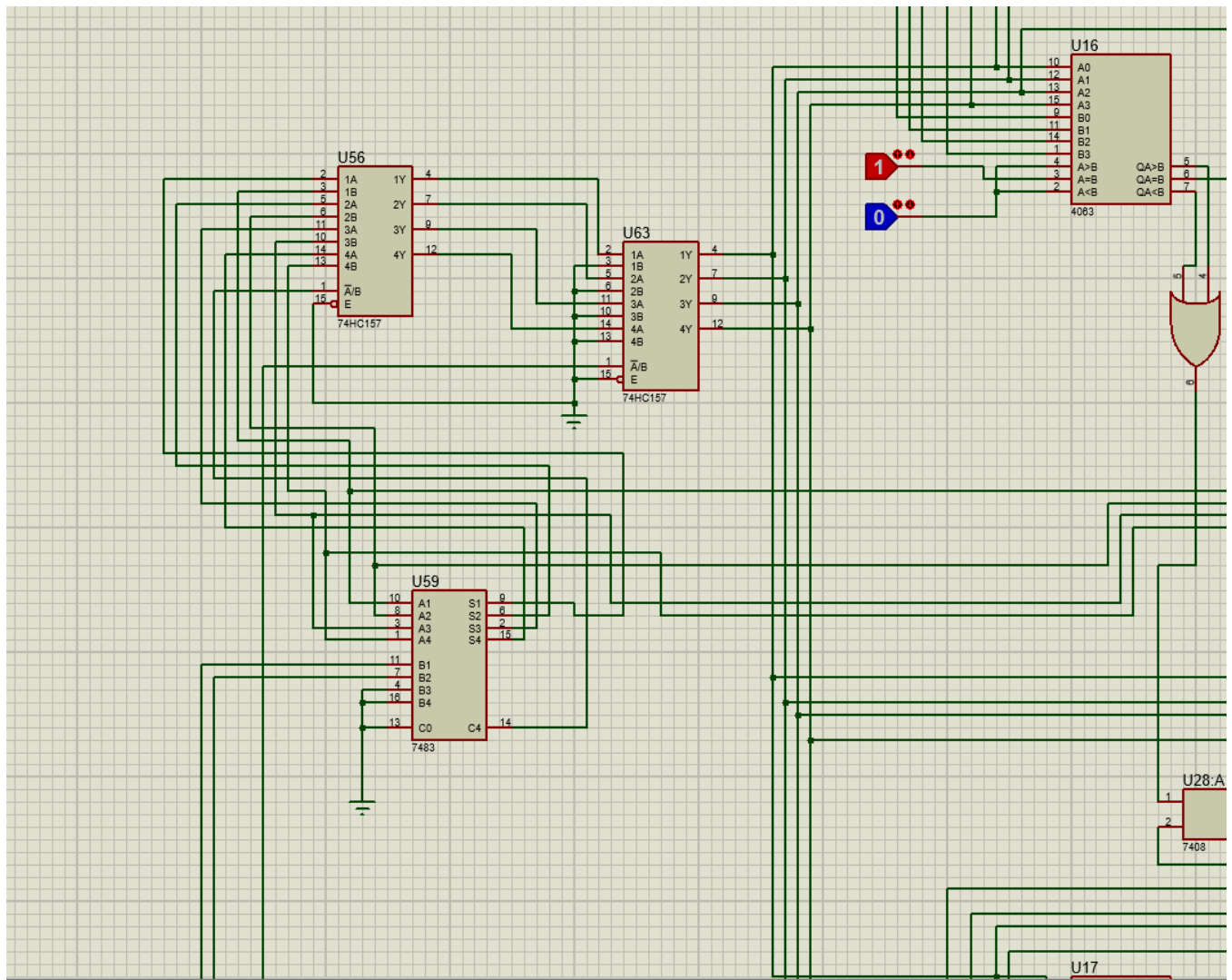


Fig: Circuits of adder & multiplexers used in the complete integration of the game circuit.

Contribution of the Team Members:

ID-220021122:

Contributed on building the **“DICE MODULE”**.

ID-220021126:

Contributed on building the **“PLAYER SELECTOR MODULE”**. Also implemented the **MULTIPLEXER** circuit to ensure that both players circuit doesn't operate at the same time.

ID-220021147:

Contributed on building the **“WINNER MODULE”** and also the **ADDER** circuit implementation so that the dice value adds up with the stored value on the memory.

ID-220021153:

Contributed on using the **“HIGH FREQUENCY MEMORY CLOCK WITH 555 TIMER IC”** and also did the **“DOCUMENTATION”**.

ID-220021154:

Contributed on building the **“MEMORY MODULE”**.

ID-220021158:

Contributed on building the **“RESET MODULE”** and also did the **“DOCUMENTATION”**.